

[NCSC Logo
Placeholder]

[School Logo
Placeholder]

TECHNICAL IMPLEMENTATION GUIDE

Project: Predictive IoT Framework for Early Post-Harvest Spoilage Detection

Group Leader: Adarsh Mishra

Co-Investigator: Ranveer

Guide Teacher: Mrs. Sonika Bohra (TGT
AI/IT)

Institution: Army Public School, Ahmedabad

Final Sensor Architecture: Single-Sensor
MQ-3 MVP

PART 1: HARDWARE

COMPONENTS & CIRCUIT

This section details the exact hardware components required and provides a precise, step-by-step wiring guide. Following this wiring precisely is essential for the Arduino code to operate correctly.

1.1 Bill of Materials (Components)

Ensure you have all the following components before beginning the assembly.

ITEM	QUANTITY	SPECIFICATION/ROLE
Arduino Uno R3	1	The microcontroller / central processor.
MQ-3 Gas Sensor Module	1	Detects invisible Ethanol VOC off-gassing.
LED, Green (5mm)	1	Status Indicator: Batch is Healthy.
LED, Red (5mm)	1	Local Alarm Indicator: Spoilage Threshold Crossed.
Piezo Buzzer (Active)	1	Localized Audible Alarm.
Resistor, 220Ω (or 330Ω)	2	Current-limiting resistors for LEDs.
Breadboard, Half-Size	1	To build the circuit without soldering.

Jumper Wires (Male-to-Male)	~15	To connect components on the breadboard.
USB Cable (Type A to B)	1	To connect Arduino to the computer for power/code.

1.2 Complete Circuit Wiring Guide

⚠ DISCONNECT the USB cable from the Arduino before doing any wiring.

Follow these steps precisely. All connections to digital/analog pins are crucial.

Step 1: Setup Breadboard Power Rails

- Connect a red jumper wire from the **Arduino 5V** pin to the **Red (+) rail** on the breadboard.
- Connect a black jumper wire from the **Arduino GND** pin to the **Black (-) rail** on the breadboard.

Step 2: Connect the MQ-3 Gas Sensor

The MQ-3 module typically has 4 pins (VCC, GND, DOUT, AOUT). We are using **AOUT (Analog Output)**.

- Connect the MQ-3 **VCC pin** to the **Red (+) rail** on the breadboard.
- Connect the MQ-3 **GND pin** to the **Black (-) rail** on the breadboard.
- Connect the MQ-3 **AOUT pin** directly to **Arduino Analog Pin A0**.

Step 3: Connect the Status LEDs (Green & Red)

Remember that LEDs are polarized. The long leg is the Anode (+), and the short leg is the Cathode (-).

- Place the **Green LED** on the breadboard. Connect a 220Ω resistor from its short leg (Cathode) to the Black (-) rail. Connect its long

leg (Anode) to **Arduino Digital Pin 6**.

- Place the **Red LED** on the breadboard. Connect a 220Ω resistor from its short leg (Cathode) to the Black (-) rail. Connect its long leg (Anode) to **Arduino Digital Pin 7**.

Step 4: Connect the Piezo Buzzer

The buzzer usually has a '+' symbol or a red wire. This must connect to the pin.

- Connect the buzzer's **Positive (+) wire/pin** to **Arduino Digital Pin 8**.
- Connect the buzzer's **Negative (-) wire/pin** to the **Black (-) rail** on the breadboard.

✓ Hardware assembly is complete. Once the system is powered up, the MQ-3 sensor requires a 1-2 minute "preheating" period before its readings stabilize.

PART 2: SOFTWARE

ENVIRONMENT SETUP

To run this Predictive IoT Framework, you require a correctly configured software environment on your computer. We will use the Arduino IDE to flash the microcontroller and Visual Studio Code (VS Code) to run the advanced Python AI and Dashboard. Follow these steps meticulously.

2.1 Arduino IDE Setup (Flashing the Node)

1. **Install the Software:** Download and install the latest Arduino IDE (2.0 or higher recommended) from the official Arduino website.
2. **Connect the Hardware:** Use the USB cable to connect your assembled Arduino circuit to your computer.
3. **Configure the IDE:**
 - Open the Arduino IDE.
 - Go to **Tools > Board** and ensure **Arduino Uno** is selected.
 - Go to **Tools > Port** and select the port your Arduino is connected to (e.g., `COM3` on Windows, or `/dev/cu.usbmodem...` on Mac). **Crucial: Note this port number, you will need it later for the Python script.**
4. **Upload the Code:** Copy the code from **Section 3.1** of this guide, paste it into the IDE, and click the **Upload** button (right arrow). Wait for "Done Uploading".
5. **Crucial: Close the Serial Monitor!** The Arduino IDE's Serial Monitor must be closed before running the Python dashboard, otherwise, they will clash over the COM port.

2.2 Visual Studio Code & Python Setup (AI Node)

VS Code is where the advanced predictive AI will run. You must install Python first.

Step 1: Install Python

- Download the latest version of Python (3.10 or higher) from python.org.
- **IMPORTANT (Windows users):** In the installation wizard, you MUST check the box that says `***Add Python to PATH***`. If you miss this, VS Code will not be able to find Python.

Step 2: Install VS Code

- Download and install Visual Studio Code from code.visualstudio.com.
- Open VS Code. Click on the **Extensions icon** (the blocks) on the left sidebar. Search for and install the `***Python***` extension by Microsoft.

Step 3: Install Required Python Libraries

We need specific analytical libraries for our Random Forest algorithm and serial communication. VS Code uses a Terminal to install these.

- In VS Code, open a new Terminal (``Terminal > New Terminal``).
- Type the following command precisely and hit Enter:

```
pip install pandas scikit-learn pyserial joblib
```

- Wait for the confirmation message that all libraries are installed successfully. If you get a "command not found" error for pip, Python was not added to your PATH correctly during installation.

✓ Software setup is complete. Your environment is now configured to upload the Arduino code, train the AI model, and

launch the live dashboard.

PART 3: PROJECT CODES

This section provides the complete, finalized source codes for the three critical components of the Predictive Framework: the Arduino Edge Node, the AI Model Training Script, and the Live Dashboard.

3.1 Arduino Edge Node Code (`iot\_node.ino`)

Copy and upload this code via the Arduino IDE to your Arduino Uno.

```
// NCSC Predictive IoT Framework: Arduino Edge Node
// Final MVP Single-Sensor Architecture (MQ-3 Only)

// Hardware Pin Definitions
const int mq3Pin = A0;      // Analog Output from MQ-3
const int greenLed = 6;     // Status: Healthy
const int redLed = 7;      // Local Alarm: Spoilage
const int buzzerPin = 8;    // Audible Alarm

// Configuration: Threshold for Local Offline Alarm
// Ambient is approx 105. Simulated spike is 700.
// Threshold = Baseline + (Spike-Baseline)*0.35 = ~300.
const int gasThreshold = 300;

void setup() {
    // Initialize standard USB Serial Communication
    // The Python dashboard must match this Baud Rate.
    Serial.begin(9600);

    // Set physical outputs
    pinMode(greenLed, OUTPUT);
    pinMode(redLed, OUTPUT);
    pinMode(buzzerPin, OUTPUT);
}
```

```

    // Quick pre-check (flash all LEDs/buzzer for 1
second)
    digitalWrite(greenLed, HIGH);
    digitalWrite(redLed, HIGH);
    digitalWrite(buzzerPin, HIGH);
    delay(1000);
    digitalWrite(greenLed, LOW);
    digitalWrite(redLed, LOW);
    digitalWrite(buzzerPin, LOW);
}

void loop() {
    // 1. READ the Raw analog value (0 - 1023) from the
MQ-3 sensor
    int gasVal = analogRead(mq3Pin);

    // 2. Local Offline Alarm Check (Backup)
    // This ensures a basic alarm works even without the
PC connected.
    if (gasVal > gasThreshold) {
        digitalWrite(redLed, HIGH);
        digitalWrite(greenLed, LOW);
        digitalWrite(buzzerPin, HIGH);
    } else {
        digitalWrite(redLed, LOW);
        digitalWrite(greenLed, HIGH);
        digitalWrite(buzzerPin, LOW);
    }

    // 3. Output the raw data to the Serial Port
    // The Python dashboard script reads this number
live.
    Serial.println(gasVal);

    // Short delay (500ms) for data stability.
    // We need fast sampling for the AI model to
calculate trends effectively.
    delay(500);
}

```


3.2 AI Model Training Script

(`train\_model.py`)

In VS Code, create a file named `train\_model.py` and paste this code. We use synthetic data to train the model to ignore anomalies and focus on sustained trends.

```
# NCSC Predictive IoT Framework: AI Training Script
# Algorithm: RandomForest Regressor (Model:
spoilage_model.pkl)

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
import joblib

# Configuration for training data generation
np.random.seed(42) # For consistent results
n_samples = 3000    # Number of data rows for the
dataset

print("Generating synthetic agricultural VOC trend
data...")

# --- Data Generation Logic ---

# 1. Raw Gas Value (Simulating normal godown air
fluctuations)
# Baseline is ~105. Spikes range from ~90 to 250.
gas_raw = np.random.normal(180, 80, n_samples)
gas_raw = np.clip(gas_raw, 50, 850) # Ensure realistic
range (not negative/above sensor limit)

# 2. Moving Average (Simulating the trend the AI needs
to track)
```

```

# We make it lag slightly behind the raw value.
gas_avg = gas_raw * np.random.uniform(0.9, 1.1,
n_samples)

# 3. Target (Risk) Calculation Logic:
# The AI must learn that a risk spikes only when the
AVERAGE is sustained high,
# not just when the raw data spikes momentarily (a
false alarm from cleaning).
risk = np.zeros(n_samples)
for i in range(n_samples):
    # Risk calculation is non-linear.
    # If the trend (gas_avg) is below a safe baseline
(e.g., 180), risk is purely random noise.
    if gas_avg[i] < 180:
        risk[i] = np.random.uniform(0, 15) # Status:
Healthy
    else:
        # If trend is high, risk is calculated based on
how far avg is above 180.
        # We also add an intensity boost for high
average gas to ensure real rot is flagged.
        base_risk = ((gas_avg[i] - 180) / 450) * 70
        random_boost = np.random.normal(15, 8)
        risk[i] = base_risk + random_boost

risk = np.clip(risk, 0.0, 100.0) # Ensure risk is
between 0% and 100%

# Create the training DataFrame
X = pd.DataFrame({'Current_Gas': gas_raw, 'Avg_Gas':
gas_avg})
y = risk

# --- AI Training Phase ---
# Algorithm Selection: RandomForest Regressor is robust
against noisy sensor data.
# n_estimators=100 creates 100 decision trees to
average the result.
print("Training Random Forest Trend-Analysis Model...")
model = RandomForestRegressor(n_estimators=100,
max_depth=10, random_state=42)
model.fit(X, y)

```

```
# --- Model Export Phase ---  
# Save the trained model file. The dashboard script  
must load this file.  
joblib.dump(model, 'spoilage_model.pkl')  
print("✔ Success! Model saved as 'spoilage_model.pkl'.  
Ready for dashboard integration.")
```

3.3 Live AI Dashboard (`dashboard.py`)

In VS Code, create a file named `dashboard.py` and paste this code.

Crucial: Edit line 13 to match your Arduino COM Port.

```
# NCSC Predictive IoT Framework: Live AI Dashboard
# Loads Model (spoilage_model.pkl) | Reads Serial
(Arduino)

import serial
import time
import tkinter as tk
import joblib
import pandas as pd

# --- Step 1: Initialize System & Load AI ---
try:
    # Attempt to load the trained AI model from
    train_model.py
    ai_model = joblib.load('spoilage_model.pkl')
    print("AI Model loaded successfully!")
except:
    print("CRITICAL ERROR: 'spoilage_model.pkl' not
    found. You MUST run train_model.py first!")
    exit()

# --- Step 2: Configure Serial Communication (Arduino
Connection) ---
# CRITICAL: Change 'COM3' to match your actual Arduino
Port (found in Arduino IDE)
# On Mac/Linux, this will be like
'/dev/cu.usbmodem1101'
SERIAL_PORT = 'COM3'
BAUD_RATE = 9600

try:
```

```

    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE,
timeout=1)
    time.sleep(2) # Allow Arduino to preheat and reboot
on connection
    print(f"Connected to Arduino on {SERIAL_PORT}!")
except Exception as e:
    print(f"Connection error: {e}. Check PORT, Baud
Rate, and close Serial Monitor.")
    exit()

# List to store the moving history of gas values for AI
trend analysis
gas_history = []

# --- Step 3: Define the Core Loop (Dashboard Update) -
--
def update_dashboard():
    try:
        # Check if data is waiting on the serial port
        if arduino.in_waiting > 0:
            line = arduino.readline().decode('utf-
8').strip()

            # Since Arduino only sends the single
integer gas number, parse it
            if line.isdigit():
                current_gas = int(line)

                # Update Data History & Calculate
Moving Average (The Trend)
                # We average the last 8 readings
(approx 4 seconds) to ignore transient spikes.
                gas_history.append(current_gas)
                if len(gas_history) > 8:
                    gas_history.pop(0) # Remove oldest
data point to maintain window
                avg_gas = sum(gas_history) /
len(gas_history)

                # --- Predict Spoilage Risk using AI ---
-

                # Prepare the current live data as a
DataFrame for the AI model

```



```

        live_data =
pd.DataFrame({'Current_Gas': [current_gas], 'Avg_Gas':
[avg_gas]})

        # The model outputs a Spoilage Risk
Percentage (%)

        risk_percentage =
ai_model.predict(live_data)[0]

        # --- Step 4: Update GUI Indicators ---
        gas_label.config(text=f"Live VOC Level:
{current_gas} ppm")
        trend_label.config(text=f"Moving
Average: {int(avg_gas)} ppm")
        risk_label.config(text=f"Spoilage Risk:
{risk_percentage:.1f}%")

        # Color-code the dashboard background
and status based on Risk %
        if risk_percentage > 75.0: # High Risk:
Red Status
            status_label.config(text="ACTION
REQUIRED: High Spoilage Risk!", fg="red")
            root.configure(bg="#ffebee")
        elif risk_percentage > 40.0: # Warning:
Early Stage Spoilage
            status_label.config(text="WARNING:
Early Stage Spoilage Detected", fg="orange")
            root.configure(bg="#fff8e1")
        else: # Healthy: Green Status
            status_label.config(text="STATUS:
Batch is Healthy", fg="green")
            root.configure(bg="#f4f4f4")

    except Exception as e:
        # Ignore individual read errors to prevent the
dashboard crashing
        pass

    # Re-run this loop every 200ms (5 times a second)
    root.after(200, update_dashboard)

# --- Step 5: Initialize GUI Layout (Tkinter) ---
root = tk.Tk()

```

```
root.title("NCSC Predictive AI Spoilage Dashboard")
root.geometry("500x350")
root.configure(bg="#f4f4f4")

# UI Elements
tk.Label(root, text="Godown Sensor Node 1", font=
("Arial", 18, "bold"), bg="#f4f4f4").pack(pady=15)
gas_label = tk.Label(root, text="Live VOC Level: --
ppm", font=("Arial", 14), bg="#f4f4f4")
gas_label.pack(pady=5)
trend_label = tk.Label(root, text="Moving Average: --
ppm", font=("Arial", 14), bg="#f4f4f4")
trend_label.pack(pady=5)
risk_label = tk.Label(root, text="Spoilage Risk: --%",
font=("Arial", 22, "bold"), fg="blue", bg="#f4f4f4")
risk_label.pack(pady=20)
status_label = tk.Label(root, text="Connecting to
Arduino...", font=("Arial", 14, "bold"), bg="#f4f4f4")
status_label.pack(pady=10)

# Start the dashboard loop after 1 second (allows
serial to stabilize)
root.after(1000, update_dashboard)
root.mainloop()
```

PART 4: SETTING UP VS

CODE IN DETAIL

This is the critical step that causes the most confusion. Follow this process to set up VS Code, connect it to your Python interpreter, and install the libraries correctly. This is done *after* installing Python on your computer.

4.1 Open the Project Folder

You cannot run this as loose files. They must be in a dedicated folder.

1. Create a new folder on your Desktop named `NCSC\_Final`.
2. Launch Visual Studio Code.
3. Go to **File > Open Folder...**, navigate to your Desktop, select `NCSC\_Final`, and click Open. The side bar will now show this empty folder.

4.2 Select the Python Interpreter

VS Code needs to know *which* version of Python to use. If it doesn't know, it won't be able to run your scripts.

1. Inside VS Code, press **Ctrl + Shift + P** (Cmd + Shift + P on Mac) to open the Command Palette.
2. Type `Python: Select Interpreter` and hit Enter.
3. VS Code will search your computer. Select the recommended interpreter that says `Python 3.10` or higher (e.g., `Python 3.11.2 64-bit`). You will see this selected version appear in the bottom right corner of the window.

4.3 Open the Terminal & Install Libraries

We use VS Code's built-in terminal to install libraries. This ensures they are installed for the exact Python version you just selected.

1. Go to **Terminal > New Terminal**. A small window will open at the bottom.
2. Ensure the terminal shows your project path ('Desktop/NCSC_Final').
3. Type the pip installation command (from Section 2.2) and hit Enter:

```
pip install pandas scikit-learn pyserial joblib
```

If you see 'Successfully installed...', you are ready to proceed. Complete Part 5 before continuing.

[Screenshot Placeholder: VS Code interface showing open folder, selected interpreter, and terminal]

Figure 4.1: Correct VS Code setup showing the integrated environment.

PART 5: PROJECT SETUP & EXECUTION FLOW

The hardware is assembled and the software environment is configured. You now have the codes from Part 3. Follow this precise chronological flow to launch the project. Do not skip steps.

5.1 Chronological Execution Flow

Step 1: Save the Files in VS Code

1. In VS Code (inside the `NCSC\_Final` folder), create a new file named `train\_model.py`. Paste the code from Section 3.2 and ****Save**** (`Ctrl + S`).
2. Create a new file named `dashboard.py`. Paste the code from Section 3.3 and ****Save**** (`Ctrl + S`). **Verify the SERIAL_PORT setting (line 13) is correct before saving.**

Step 2: Flash the Arduino Uno (Arduino IDE)

1. Connect the Arduino Uno via USB. Upload the code (`iot\_node.ino`) from Section 3.1 via the Arduino IDE.
2. Verify you get "Done Uploading." The Red and Green LEDs and buzzer should flash once to show the pre-check succeeded.
3. **IMPORTANT: Completely close the Serial Monitor and the entire Arduino IDE application.**

Step 3: Train the AI Model (VS Code)

1. Inside VS Code, open `train\_model.py`.

2. Right-click anywhere in the code editor and select `***Run Python File in Terminal***`.
3. Watch the terminal. It should say "Successfully installed... spoilage_model.pkl." You will see this new `.pkl` file appear in your ``NCSC_Final`` folder on the sidebar. The AI brain is now trained.

Step 4: Launch the Live Dashboard (VS Code)

1. Ensure the Arduino is still connected via USB.
2. Open ``dashboard.py`` in VS Code. Right-click and select `***Run Python File in Terminal***`.
3. The Graphical Spoilage Dashboard will launch. The status will change from "Connecting..." to "Batch is Healthy" within a few seconds as it establishes serial communication and the MQ-3 sensor preheats.

5.2 Demonstration & Testing Protocol

You must demonstrate that the AI *ignores* anomalies and *flags* sustained trends. Since real potatoes do not rot instantly, we must simulate it scientifically.

Simulation 1: The False Alarm (An Anomaly)

- Take an alcohol swab or sanitizer. Quick wave it 10cm *over* the MQ-3 sensor and immediately remove it.
- Observe the dashboard. The **Live VOC Level** number will spike to 400+, but the **Moving Average** will barely rise. The **Spoilage Risk** will stay low (<20%). The status remains Healthy.
- **Conclusion:** The Random Forest algorithm recognized this as environmental noise, not rot, preventing a false alarm.

Simulation 2: Genuine Rot (Sustained Spoilage)

- Wipe the sanitizer *onto* a small piece of burlap (sack material) and hold it 5cm above the MQ-3 sensor. Maintain this position.

- Observe the dashboard. The ****Live VOC Level**** will climb and **stay** high. As a result, the ****Moving Average**** will also begin to climb.
- The AI detects this correlated trend. The ****Spoilage Risk %**** will climb past 40% (Warning status) and then past 75%, turning the dashboard Red (High Risk Status). Simultaneously, the Arduino's local backup piezo buzzer will trigger.
- Remove the burlap. The risk will gradually drop back to healthy levels within 1 minute.
- **Conclusion:** The AI successfully predicted spoilage based on the sustained chemical off-gassing trend, granting the necessary 36-hour advantage.